PES-VCS Lab Report

Name - Kotta Tharith

Reddy

USN: PES1UG24CS233

Course: PES VCS Lab

Phase 1: Object Storage

Command: make test_objects

Output:

```
remote: Enumerating objects: 17, done.
remote: Counting objects: 100% (17/17), done.
remote: Compressing objects: 100% (17/17), done.
remote: Total 17 (delta 0), reused 11 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (17/17), 24.80 KiB | 127.00 KiB/s, done.
```

Phase 2: Tree Objects

Command: make test_tree

Output:

```
make
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
```

Phase 3: Index (Staging Area)

Commands:

./pes init

./pes add file1.txt file2.txt
./pes status

Output:

```
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Phase 4: Commit and Log

Commands:

./pes commit -m "Initial commit"

./pes log

Output:

```
index.c:225:5: warning: ignoring return value of ‘fread’ declared with
  225 |     fread(data, 1, size, f);
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Phase 5:

Output:

```
index.c:225:5: warning: ignoring return value of ‘fread’ declared with
  225 |     fread(data, 1, size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -o test_objects test_objects.o object.o -lcrypto
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Phase 6

Output:

```
cat .pes/HEAD
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Committed as a53d53d7831f2adbf44839d85b61e9e4cc159a2e190ba313fb9cf37c3fc8991b
Committed: a53d53d7831f2adbf44839d85b61e9e4cc159a2e190ba313fb9cf37c3fc8991b
a53d53d7831f2adbf44839d85b61e9e4cc159a2e190ba313fb9cf37c3fc8991b
```

Analysis Questions

Q5.1:A branch in PES-VCS is a file in .pes/refs/heads/ containing a commit hash.

To implement pes checkout <branch>, the following steps are required:

1. Update HEAD

- Modify .pes/HEAD to:modified files.

- This makes HEAD point to the selected branch.

2. Read the target commit

- Read the commit hash from .pes/refs/heads/<branch>
- Load the commit object and retrieve its tree hash

3. Update the working directory

- Traverse the tree object recursively
- For each file (blob):
- Write its contents to the working directory
- Remove files that exist in the current working directory but not in the target

tree

4. Update the index

- Rebuild .pes/index to match the checked-out tree

Q5.2:Before checkout, we must ensure no uncommitted changes will be lost.

Steps:

1. For each file in the index:

- Compare stored metadata (mtime, size) with the working directory
- If different → file is modified

2. Recompute the file's hash:

- Compare with the hash stored in the index
- If mismatch → file content changed

3. Compare with target branch tree:

- If a file differs between current state and target branch AND
- The working directory has uncommitted changes → conflict

Q5.3:A detached HEAD occurs when .pes/HEAD contains a commit hash directly instead of a branch reference.

What happens:

- New commits can still be created
- Each new commit points to the previous commit
- BUT no branch points to these commits

These commits are not reachable from any branch

Q6.1:o remove unreachable objects:

Algorithm:

1. Start from all root references:
- .pes/refs/heads/* (branches)
- HEAD
2. Traverse object graph:
- Commit → Tree → Blob
- Follow parent commits recursively
3. Use a Hash Set to track visited objects
4. After traversal:
- Any object not in the set = unreachable
5. Delete unreachable objects from .pes/objects/

Data Structure:

- Use a hash set (or unordered_set) for fast lookup (O(1))

Scale Estimation:

- 100,000 commits + 50 branches
- Many commits share trees/blobs

Actual objects visited ≈ 100k–300k objects

Q6.2:Race Condition:

1. GC starts and scans reachable objects

2. A new commit is being created:

- Objects written but not yet referenced by a branch

3. GC does not see these objects as reachable

4. GC deletes them

5. Commit finishes → now references missing objects → corruption

---

How Git avoids this:

- Locks references during updates
- Uses atomic operations (rename)
- Keeps recent objects protected temporarily
- Uses mark-and-sweep safely after commit completio